

Technische Hochschule Deggendorf
Faculty Applied Information Technology
Studiengang Bachelor of Cyber-Security

Damn Vulnerable Drone Supply Chain Compromise - Project Docs

Vorgelegt von:

Christof Renner
Matrikelnummer: 22301943
Manuel Friedl
Matrikelnummer: 1236626
Felix Hahl
Matrikelnummer:

Prüfungsleitung:

Prof. Dr. Michael Heigl

Am: 17. Juli 2025

Contents

1	Einleitung und Projektübersicht	3
1.1	Zielsetzung	3
1.2	Projektkontext	3
2	Technische Architektur	3
2.1	Gesamtarchitektur	3
2.2	Netzwerkstruktur	4
3	Entwicklungsprozess	4
3.1	Konzeption und Planung	4
3.2	Technische Implementierung	5
3.3	Sicherheitslücken und Angriffspfad	5
4	Arbeitsaufteilung	6
5	Technische Herausforderungen und Lösungen	7
5.1	Blockers: Schwierigkeiten mit virtuellen Maschinen	7
5.2	Blockers: Performance bei Start der Container	7
5.3	Blockers: Sourceanpassung beim Container Rebuild	7
6	Testergebnisse und Evaluation	8
6.1	Testmethodik	8
6.2	Fehlerbehandlung in Docker-Containern	8
6.3	Feedback und Iterationen	8
7	Fazit und Ausblick	9
7.1	Projektergebnisse	9
7.2	Lehren aus dem Projekt	9
7.3	Abschließende Bemerkungen	9

1 Einleitung und Projektübersicht

Dieses Dokument beschreibt die Entwicklung und technische Umsetzung des „Damn Vulnerable Drone Supply Chain Compromise“ Projekts im Rahmen des Moduls "Wahlpflichtfach Projekt". Dabei wird auf dem originalen Projekt "Damn Vulnerable Drone" von Nicholas Aleks aufgebaut.

1.1 Zielsetzung

Das Hauptziel dieses Projekts bestand darin, eine realistische Simulationsumgebung zu schaffen, in der Studierende praxisnah die Konzepte und Techniken von Supply-Chain-Angriffen erlernen und anwenden können. Dabei wurden folgende spezifische Lernziele definiert:

- Verständnis der Grundlagen von Supply-Chain-Angriffen in der Softwareentwicklung
- Praktische Anwendung von Reconnaissance-Techniken in Linux-Umgebungen
- Identifikation und Ausnutzung von Schwachstellen in Build-Prozessen
- Implementierung von Supply-Chain-Angriffen zur Kompromittierung von Drohnensoftware
- Analyse der Angriffstechniken im Kontext des MITRE ATT&CK-Frameworks

1.2 Projektkontext

Wie bereits angesprochen basiert unser Projekt auf dem Open-Source-Projekt „Damn Vulnerable Drone“ von Nicholas Aleks, das eine virtuelle Drohenumgebung für Sicherheitstests bietet. Für unsere Zwecke haben wir dieses Projekt erweitert und ein spezifisches CTF-Szenario entwickelt, das auf Supply-Chain-Angriffe fokussiert ist. Die Herausforderung besteht darin, durch Ausnutzung einer Sicherheitslücke im Entwicklungsprozess einer simulierten Drohnensoftwarefirma, eine Drohne zum Absturz zu bringen.

2 Technische Architektur

2.1 Gesamtarchitektur

Die Simulationsumgebung wurde vollständig mit Docker-Containern realisiert, um eine konsistente und isolierte Lernumgebung zu gewährleisten. Die Hauptkomponenten des Systems sind:

- **Entwickler-Maschine:** Simuliert die Arbeitsumgebung eines Entwicklers bei einer Drohnensoftwarefirma.
- **Ground Control Station (GCS):** Steuert die Drohne und enthält das zu kompromittierende "Return-to-Land"-Modul.
- **Companion Computer:** Simuliert den Bordcomputer der Drohne für fortgeschrittene Funktionen.

-
- **Flight Controller:** Repräsentiert die eigentliche Flugsteuerung der Drohne, basierend auf ArduPilot-Firmware.
 - **Simulator:** Gazebo-basierte 3D-Umgebung zur Simulation der Drohnenphysik und -interaktion.

2.2 Netzwerkstruktur

Die Container sind in zwei Netzwerken organisiert:

- **Infrastrukturnetzwerk (10.13.0.0/24):** Verbindet alle Container miteinander und ermöglicht die grundlegende Kommunikation zwischen den Systemkomponenten.
- **Simuliertes WLAN (192.168.13.0/24):** Simuliert die drahtlose Verbindung zwischen der Ground Control Station und dem Companion Computer. Dieses Netzwerk wird virtuell über Container-Links implementiert.

3 Entwicklungsprozess

3.1 Konzeption und Planung

Die Entwicklung des CTF-Szenarios begann mit einer detaillierten Konzeptphase. Zuerst analysierten wir die ursprüngliche Damn Vulnerable Drone-Plattform, um ein Verständnis für ihre Funktionsweise und Möglichkeiten zu entwickeln. Anschließend definierten wir das Szenario eines Supply-Chain-Angriffs, bei dem ein Angreifer Zugriff auf die Entwicklungsumgebung einer Drohnensoftwarefirma erhält und versucht, schädlichen Code in den Build-Prozess einzuschleusen.

Wir legten folgende Designprinzipien fest:

- **Realismus:** Das Szenario sollte realistische Schwachstellen widerspiegeln, die in tatsächlichen Entwicklungsumgebungen vorkommen könnten.
- **Lernwert:** Jeder Schritt der Challenge sollte wichtige Sicherheitskonzepte vermitteln.
- **Zugänglichkeit:** Die Challenge sollte für Studierende mit grundlegenden Linux-Kenntnissen lösbar sein.
- **Skalierbarkeit:** Die Umgebung sollte auf verschiedenen Systemen lauffähig sein, mit minimalen Anforderungen.

3.2 Technische Implementierung

Die Implementierung erfolgte in mehreren Phasen:

1. **Container-Setup:** Erstellung der Docker-Container für die verschiedenen Komponenten des Systems. Wir verwendeten Alpine Linux als Basis für die meisten Container, um die Größe zu minimieren.
2. **Integration der Drohnensimulation:** Konfiguration von ArduPilot SITL (Software-in-the-Loop) und Gazebo für die realistische Simulation des Drohnenverhaltens.
3. **Implementierung der Build-Pipeline:** Entwicklung der automatisierten Build- und Deployment-Prozesse, die als Angriffsziel dienen. Dies umfasste die Erstellung von Cron-Jobs und Bash-Skripten.
4. **Implementierung der Schwachstellen:** Gezielte Integration von Sicherheitslücken, insbesondere im Zusammenhang mit Berechtigungen und unsicheren Automatisierungsprozessen.
5. **Entwicklung des Web-Interfaces:** Erstellung einer benutzerfreundlichen Weboberfläche zur Steuerung der Simulation und Visualisierung des Drohnenverhaltens.
6. **Dokumentation und Walkthrough:** Erstellung detaillierter Anleitungen sowohl für die Setup-Phase als auch für die Lösung der Challenge.

3.3 Sicherheitslücken und Angriffspfad

Der Hauptangriffspfad der Challenge beinhaltet folgende Schritte:

1. **Initiale Zugriff:** Teilnehmer erhalten Zugang zur Entwickler-Maschine mit den Anmeldedaten eines Junior-Entwicklers (developer:dev123).
2. **Reconnaissance:** Erkundung der Umgebung, um Systemstrukturen und potenzielle Angriffspunkte zu identifizieren.
3. **Entdeckung der Build-Pipeline:** Identifikation des automatisierten Build-Prozesses und seiner Komponenten.
4. **Ausnutzung von Cron-Jobs:** Analyse der als Root ausgeführten Cron-Jobs und Identifikation der Build-Update-Skripte.
5. **Code-Injektion:** Modifikation des Quellcodes oder der Docker-Dateien, um schädlichen Code in den Build-Prozess einzuschleusen.
6. **Persistence:** Warten auf die automatische Ausführung des Build-Prozesses und Deployment des modifizierten Codes.
7. **Impact:** Auslösen des "Return-to-Land"-Befehls, der nun durch den injizierten Code die Drohne zum Absturz bringt.

Diese Schritte entsprechen mehreren Taktiken des MITRE ATT&CK-Frameworks, darunter Initial Access, Discovery, Persistence, Privilege Escalation, Defense Evasion und Impact.

4 Arbeitsaufteilung

Die Entwicklung des Projekts wurde auf mehrere Teammitglieder aufgeteilt, wobei jedes Mitglied spezifische Verantwortungsbereiche übernahm. Die folgende Tabelle zeigt die detaillierte Arbeitsaufteilung:

Teammitglied	Hauptaufgaben	Spezifische Aufgaben
Christof Renner	Technische Architektur & Docker-Integration	• Entwicklung der Container-Architektur • Konfiguration der verschiedenen Docker-Container • Ausarbeitung des Pipeline Compromise • Ausarbeitung des Walk-throughs
Manuel Friedl	Dokumentation & Start-Skript	• Ausarbeitung des Walk-throughs • Konzeptionierung des Szenarios • Ausarbeitung der start.sh Skriptlogik • Erstellung der Simulationsszenarien
Felix Hahl	Dokumentation & Testing	• Erstellung der technischen Dokumentation • Entwicklung des Walk-throughs • Qualitätssicherung und Testing • Konzeptionierung des Szenarios

Dabei wurde in regelmäßige Abstimmungen und gemeinsame Review-Sessions sichergestellt, dass alle Komponenten nahtlos zusammenarbeiten.

5 Technische Herausforderungen und Lösungen

5.1 Blockers: Schwierigkeiten mit virtuellen Maschinen

Problemstellung: Während der Entwicklung des Projekts stießen wir auf signifikante technische Herausforderungen im Zusammenhang mit der Ausführung in virtuellen Maschinen. Das ursprüngliche Projekt war für eine Kali-Linux VM konzipiert, was zu Problemen führte, da für den Start der ersten Phase der Drohnensimulation eine GPU erforderlich war, die in der VM nicht verfügbar war. Aufgrund verschiedener Treiberprobleme gestaltete sich ein GPU-Passthrough als schwierig.

Lösungsansatz: Nach weiterem Testing wurde festgestellt, dass die Simulation ebenso auf Barebone-Systemen lauffähig war. Diese Erkenntnis machte die Abhängigkeit von einer GPU in der VM überflüssig und ermöglichte einen flexibleren Einsatz der Simulationsumgebung auf verschiedenen Systemkonfigurationen.

5.2 Blockers: Performance bei Start der Container

Problemstellung: Ein weiteres Problem trat beim Start der Container auf. Da mehrere Container gleichzeitig gestartet wurden, führte dies zu einer hohen Systemlast und Performanceeinbußen, besonders auf Systemen mit begrenzten Ressourcen.

Lösungsansatz: Dieses Problem konnte nur teilweise gelöst werden. Da im Rahmen des Supply-Chain-Angriffs die Container der Drohne neu gestartet werden müssen, wurde ein weitgehendes Caching implementiert. Diese Optimierung reduzierte die Startzeit und die benötigte Rechenleistung erheblich, ohne die Funktionalität der Simulation zu beeinträchtigen.

5.3 Blockers: Sourceanpassung beim Container Rebuild

Problemstellung: Ein weiteres Problem trat bei der Sourceanpassung auf. Hier bestand die Schwierigkeit darin, dass die Container beim Rebuild nicht die angepasste Source verwendeten, sondern auf die Original-Source zurückgriffen. Dies verhinderte effektiv die Simulation des Supply-Chain-Angriffs, da modifizierter Code nicht in den Build-Prozess einfließen konnte.

Lösungsansatz: Diese Herausforderung wurde durch eine Anpassung der Dockerfile und der Start-Skripte gelöst. Die Änderungen stellten sicher, dass die angepasste Source immer verwendet wird und beim Rebuild korrekt in die Container integriert wird. Dadurch konnte der Supply-Chain-Angriffspfad wie vorgesehen implementiert werden.

6 Testergebnisse und Evaluation

6.1 Testmethodik

Um die grundlegende Funktionalität der CTF-Challenge zu validieren, wurden interne Reviews durchgeführt:

- **Explorative Tests:** Manuelle Durchführung des vorgesehenen Angriffspfadens zur Verifizierung der Funktionalität.
- **Ad-hoc Usability-Überprüfung:** Informelle Bewertung der Benutzerführung durch Teammitglieder.
- **Ressourcenverbrauch-Beobachtung:** Allgemeine Überwachung der Systemauslastung während des Betriebs.
- **Betriebssystem-Kompatibilität:** Grundlegende Überprüfung auf Ubuntu- und Windows-Systemen mit Docker-Installation.

Die Testphase wurde dabei bewusst iterativ gestaltet, um schnell auf beobachtete Probleme reagieren zu können, ohne einen formalisierten Testplan zu implementieren.

6.2 Fehlerbehandlung in Docker-Containern

Zur Verbesserung der Stabilität wurden verschiedene Fehlerbehandlungsmechanismen implementiert:

- **Container-Gesundheitsüberwachung:** Implementierung von Docker-Healthchecks zur automatischen Erkennung von Fehlzuständen.
- **Automatische Neustarts:** Konfiguration der Container mit Restart-Policies (restart: unless-stopped), um bei Abstürzen selbstständig neu zu starten.
- **Graceful Degradation:** Implementierung von Fallback-Mechanismen, die bei Ausfall nicht-kritischer Komponenten einen eingeschränkten Betrieb ermöglichen.
- **Verbesserte Startskripte:** Ergänzung der Docker-Compose-Konfiguration um Abhängigkeitsbeziehungen und Wartezeiten, um Race-Conditions beim Systemstart zu vermeiden.

6.3 Feedback und Iterationen

Basierend auf internem Feedback wurden mehrere Verbesserungen vorgenommen:

- **Verbesserte Dokumentation:** Ergänzung der Anleitungen mit spezifischeren Erläuterungen zu den Supply-Chain-Angriffsvektoren.
- **Performance-Optimierungen:** Reduzierung der Container-Baseline-Images und Entfernung nicht benötigter Dienste zur Verringerung des Ressourcenverbrauchs.
- **Resiliente Netzwerkkonfiguration:** Verbesserte Netzwerkeinstellungen, um mit temporären Verbindungsunterbrechungen zwischen Containern besser umgehen zu können.

7 Fazit und Ausblick

7.1 Projektergebnisse

Das "Damn Vulnerable Drone Supply Chain Compromise"-Projekt wurde erfolgreich entwickelt und implementiert. Es bietet eine realistische und lehrreiche Umgebung, in der Studierende praktische Erfahrungen mit Supply-Chain-Angriffen sammeln können. Die wichtigsten Errungenschaften des Projekts sind:

- Eine vollständig funktionsfähige Drohnensimulationsumgebung, die realistische Angriffsszenarien ermöglicht.
- Ein strukturierter Lernpfad mit klaren Lernzielen und praktischen Übungen.
- Eine detaillierte Dokumentation, die sowohl den Aufbau der Umgebung als auch die Lösungswege erläutert.
- Eine optimierte Containerarchitektur, die auf verschiedenen Systemen lauffähig ist.

7.2 Lehren aus dem Projekt

Während der Entwicklung haben wir wichtige Erkenntnisse gewonnen:

- Die Balance zwischen Realismus und Zugänglichkeit ist entscheidend für den Erfolg einer Lernumgebung.
- Containerisierung bietet erhebliche Vorteile für die Erstellung konsistenter und portabler Lernumgebungen.
- Die Leistungsoptimierung ist ein kritischer Faktor, insbesondere für rechenintensive Simulationen.

7.3 Abschließende Bemerkungen

Das Damn Vulnerable Drone Supply Chain Compromise Projekt demonstriert die Wichtigkeit praktischer, realitätsnaher Übungen in der Cybersecurity-Ausbildung. Durch die Kombination von Drohnentechnologie, containerisierter Virtualisierung und realistischen Angriffsszenarien bietet es eine wertvolle Lernumgebung für angehende Sicherheitsexperten. Die gewonnenen Erkenntnisse und entwickelten Techniken können auch auf andere Bereiche der IT-Sicherheitsausbildung übertragen werden.